

```
In [3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [6]: #pip install scipy
```

```
In [2]: import scipy
print(scipy.__version__)
```

1.14.0

```
In [6]: from scipy.optimize import minimize
```

```
In [7]: def f(x):
        return (x-3)**2
res= minimize(f,2)
```

```
In [8]: res
```

```
Out[8]: message: Optimization terminated successfully.
        success: True
        status: 0
         fun: 5.551437397369767e-17
          x: [ 3.000e+00]
         nit: 2
         jac: [-4.325e-13]
        hess_inv: [[ 5.000e-01]]
         nfev: 6
         njev: 3
```

```
In [9]: res.x[0]
```

```
Out[9]: np.float64(2.999999992549203)
```

```
In [10]: res.x
```

```
Out[10]: array([2.99999999])
```

Optimizer are helps to find the minimum value of a function or root of an equation

```
In [ ]: # Minimizing the function of data:
        # high points are define as maxima and low funcation call minima
        # for findinf the minima
```

```
In [41]: from scipy.optimize import minimize
def eqn(x):
    return x**2 + x + 2
```

```
mymin= minimize(eqn, 0, method='BFGS')
print(mymin)
```

```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 1.75
   x: [-5.000e-01]
  nit: 2
  jac: [ 0.000e+00]
hess_inv: [[ 5.000e-01]]
 nfev: 8
 njev: 4
```

```
In [42]: from scipy.optimize import minimize
def eqn(x):
    return x**2 + x + 2
mymin= minimize(eqn, 0, method='CG')
print(mymin)
```

```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 1.75
   x: [-5.000e-01]
  nit: 2
  jac: [ 2.980e-08]
 nfev: 10
 njev: 5
```

How CG Works:--

It's a modification of the steepest descent algorithm.

Instead of just using gradient directions, it uses conjugate directions to make optimization faster (especially for quadratic functions).

CG requires the objective function to be differentiable.

```
In [46]: from scipy.optimize import minimize
def eqn(x):
    return x**2 + x + 2
```

```
mymin= minimize(eqn, 0, method='TNC')
print(mymin)
```

```
message: Local minimum reached (|pg| ~= 0)
success: True
status: 0
  fun: 1.75
   x: [-5.000e-01]
  nit: 1
  jac: [ 2.220e-08]
 nfev: 8
```

TNC stands for Truncated Newton Conjugate-Gradient algorithm.

It is used for minimizing scalar functions subject to bounds on variables, but no constraints.

```
In [48]: from scipy.optimize import minimize
def eqn(x):
    return x**2 + x + 2
mymin= minimize(eqn, 0, method='COBYLA')
print(mymin)
```

```
message: Optimization terminated successfully.
success: True
status: 1
  fun: 1.750000001
   x: [-5.001e-01]
  nfev: 18
 maxcv: 0.0
```

COBYLA (Constrained Optimization BY Linear Approximations) is a gradient-free method (good for functions without derivatives).

```
In [49]: from scipy.optimize import minimize
def eqn(x):
    return x**2 + x + 2
mymin= minimize(eqn, 0, method='SLSQP')
print(mymin)
```

```
message: Optimization terminated successfully
success: True
status: 0
  fun: 1.75
    x: [-5.000e-01]
  nit: 2
  jac: [ 1.490e-08]
 nfev: 5
 njev: 2
```

In []:

In []:

scipy cluster:

```
In [63]: import numpy as np
from scipy.cluster.vq import kmeans, whiten
import seaborn as sns
sns.set_style("darkgrid")

#create 50 datapoints in two cluster
pts=100
a=np.random.multivariate_normal([0,0], [[4,1],[1,4]], size=pts)
b=np.random.multivariate_normal([30,10], [[10,2],[2,1]], size=pts)
features= np.concatenate((a,b))

#whiten data
whitened= whiten(features)

#find 2 clusters in the data
codebook, distortion= kmeans(whitened,2)

#plt for whiten data and clusters centers in red
plt.scatter(whitened[:,0], whitened[:,1])
plt.scatter(codebook[:,0], codebook[:,1], c='r')
plt.show()
```



`scipy.cluster.vq` provides vector quantization tools: `kmeans` for clustering. & `whiten` for feature scaling.

Whitening means scaling each feature to unit variance.

This helps in clustering algorithms like KMeans, which are sensitive to feature scale.

After this, all features contribute equally regardless of original units.

In []:

codebook contains the centroids
(cluster centers) → shape will be (2, 2)

distortion is the sum of squared
distances from each point to its closest
center (a measure of how good the
clustering is).

scipy constant

```
In [21]: import scipy
print("scipy --golden ratio value= %.18f"%scipy.constants.golden)
```

```
scipy --golden ratio value= 1.618033988749894903
```

```
In [12]: from scipy import constants
```

```
In [13]: print("speed of light", constants.c, "m/s")
```

```
speed of light 299792458.0 m/s
```

```
In [14]: print("Planck's constant:", constants.h)
```

```
Planck's constant: 6.62607015e-34
```

```
In [15]: from scipy.constants import find
find('boltzman')
```

```
Out[15]: ['Boltzmann constant',
          'Boltzmann constant in Hz/K',
          'Boltzmann constant in eV/K',
          'Boltzmann constant in inverse meter per kelvin',
          'Stefan-Boltzmann constant']
```

```
In [20]: print(constants.yotta)
print(constants.zetta)
print(constants.exa)
print(constants.peta)
print(constants.tera)
print(constants.giga)
print(constants.mega)
print(constants.kilo)
print(constants.hecto)
print(constants.deka)
print(constants.deci)
print(constants centi)
print(constants.milli)
print(constants.nano)
```

```
print(constants.micro)
print(constants.femto)
print(constants.atto)
print(constantszepto)
```

```
1e+24
1e+21
1e+18
10000000000000000.0
100000000000000.0
10000000000.0
1000000.0
1000.0
100.0
10.0
0.1
0.01
0.001
1e-09
1e-06
1e-15
1e-18
1e-21
```

BINARY PREFIXS:

```
In [22]: print(constants.kibi)
print(constants.mebi)
print(constants.gibi)
print(constants.tebi)
print(constants.pebi)
print(constants.exbi)
print(constants.zebi)
print(constants.yobi)
```

```
1024
1048576
1073741824
1099511627776
1125899906842624
1152921504606846976
1180591620717411303424
1208925819614629174706176
```

MASS VALUE:

```
In [23]: print(constants.gram)
print(constants.metric_ton)
print(constants.grain)
print(constants.lb)
print(constants.pound)
print(constants.oz)
print(constants.long_ton)
```

```
print(constants.short_ton)
print(constants.ounce)
print(constants.troy_ounce)
print(constants.troy_pound)
print(constants.carat)
print(constants.atomic_mass)
```

```
0.001
1000.0
6.479891e-05
0.45359236999999997
0.45359236999999997
0.028349523124999998
1016.0469088
907.1847399999999
0.028349523124999998
0.031103476799999998
0.37324172159999996
0.0002
1.6605390666e-27
```

degree value:

```
In [25]: print(constants.degree)
print(constants.arcmin)
print(constants.arcsec)
print(constants.arcminute)
print(constants.arcsecond)
```

```
0.017453292519943295
0.0002908882086657216
4.84813681109536e-06
0.0002908882086657216
4.84813681109536e-06
```

time value

```
In [27]: print(constants.minute)
print(constants.hour)
print(constants.day)
print(constants.week)
print(constants.year)
print(constants.Julian_year)
```

```
60.0
3600.0
86400.0
604800.0
31536000.0
31557600.0
```

length value:


```
In [29]: print(constants.inch)
print(constants.foot)
print(constants.yard)
print(constants.mile)
print(constants.mil)
print(constants.pt)
print(constants.point)
print(constants.survey_foot)
print(constants.nautical_mile)
print(constants.fermi)
print(constants.angstrom)
print(constants.micron)
print(constants.au)
print(constants.astronomical_unit)
print(constants.light_year)
```

```
0.0254
0.30479999999999996
0.9143999999999999
1609.3439999999998
2.5399999999999997e-05
0.0003527777777777776
0.0003527777777777776
0.3048006096012192
1852.0
1e-15
1e-10
1e-06
149597870700.0
149597870700.0
9460730472580800.0
```

precure value- pascals:

```
In [32]: print(constants.atm)
print(constants.atmosphere)
print(constants.bar)
print(constants.torr)
print(constants.mmHg)
print(constants.psi)
```

```
101325.0
101325.0
100000.0
133.32236842105263
133.32236842105263
6894.757293168361
```

volume- cubic meter:

```
In [34]: print(constants.liter)
print(constants.liter)
print(constants.gallon)
print(constants.gallon_US)
print(constants.gallon_imp)
print(constants.fluid_ounce)
print(constants.fluid_ounce_US)
print(constants.fluid_ounce_imp)
print(constants.barrel)
print(constants.bbl)
```

```
0.001
0.001
0.0037854117839999997
0.0037854117839999997
0.00454609
2.9573529562499998e-05
2.9573529562499998e-05
2.84130625e-05
0.15898729492799998
0.15898729492799998
```

speed value:(meter per sec)

```
In [35]: print(constants.kmh)
print(constants.mph)
print(constants.mach)
print(constants.speed_of_sound)
print(constants.knot)
```

```
0.2777777777777778
0.44703999999999994
340.5
340.5
0.5144444444444445
```

temperature: kelvin value

```
In [36]: print(constants.degree_Fahrenheit)
print(constants.zero_Celsius)
```

```
0.5555555555555556
273.15
```

energy- joules

```
In [39]: print(constants.eV)
print(constants.electron_volt)
print(constants.calorie)
print(constants.calorie_th)
print(constants.calorie_IT)
```

```
print(constants.erg)
print(constants.Btu)
print(constants.Btu_IT)
print(constants.Btu_th)
print(constants.ton_TNT)
```

```
1.602176634e-19
1.602176634e-19
4.184
4.184
4.1868
1e-07
1055.05585262
1055.05585262
1054.3502644888888
4184000000.0
```

In []:

In []:

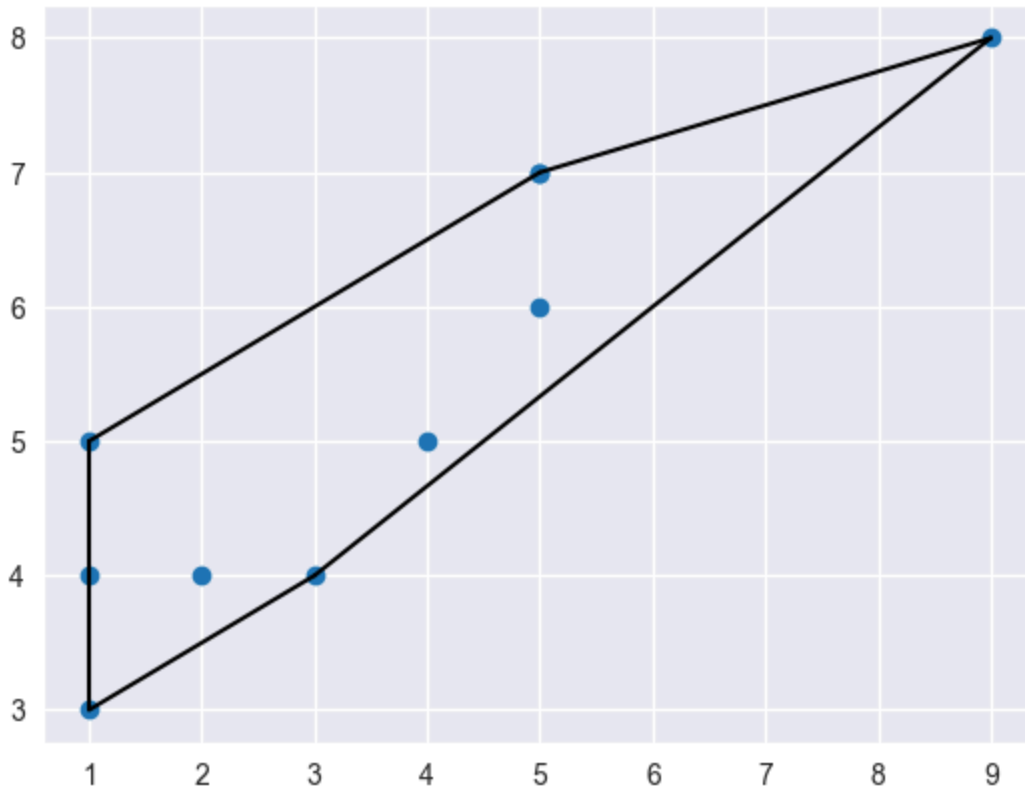
scipy FFTPACK:

```
In [78]: import numpy as np
from scipy.cluster.vq import kmeans, whiten
import seaborn as sns
sns.set_style("darkgrid")

#frequency in terms of herz.
fre= 10

#sample_rate
fre_samp= 100

t=np.linspace(0,2,2*fre_samp, endpoint= False)
a=np.sin(fre *2* np.pi *t )
plt.plot(t,a)
plt.xlabel('Time(s)')
plt.ylabel('signal amplitude')
plt.show()
```

`ConvexHull(new)`: It Calculates the smallest convex polygon (convex hull) that contains all the points.

`hull.simplices`: Returns the vertex indices that form the edges of the convex hull.

`scipy-graph`:

whenever we working with graph we have module from `scipy`.

In this we also have- an adjacency metrix: $n \times n$ matrix where n is the number of elements in the graph

```
In [56]: import numpy as np
from scipy.sparse.csgraph import connected_components
from scipy.sparse import csr_matrix
new_value= np.array([[1,2,3],[1,0,0], [2,3,4]])
value_new= csr_matrix(new_value)
print(connected_components(value_new))
```

```
(1, array([0, 0, 0], dtype=int32))
```

```
In [57]: import numpy as np
from scipy.sparse.csgraph import connected_components
from scipy.sparse import csr_matrix #  Correct import

# Sample adjacency matrix (should be square and represent a graph)
new_value = np.array([
    [0, 1, 1],
    [1, 0, 0],
    [1, 0, 0]
])

# Convert to sparse matrix
value_new = csr_matrix(new_value)

# Compute connected components
n_components, labels = connected_components(value_new)

print("Number of connected components:", n_components)
print("Component labels for each node:", labels)
```

```
Number of connected components: 1
Component labels for each node: [0 0 0]
```

for dijkstra algo - we find the shortest path using this from one element to another and for this its take 3 argu. limit, indices, return_predecessors

```
In [60]: import numpy as np
from scipy.sparse.csgraph import dijkstra
from scipy.sparse import csr_matrix
new_value= np.array([[1,2,3],[1,0,0], [2,3,4]])
value_new= csr_matrix(new_value)
print(dijkstra(value_new, return_predecessors=True, indices=0))
```

```
(array([0., 2., 3.]), array([-9999,      0,      0], dtype=int32))
```

floyd warshall- In this also we use and we find the shortest path using between all the pairs of elements

```
In [67]: import numpy as np
from scipy.sparse.csgraph import floyd_warshall
from scipy.sparse import csr_matrix
new_value= np.array([[1,2,3],[1,0,0], [2,3,4]])
value_new= csr_matrix(new_value)
print(floyd_warshall(value_new, return_predecessors=True))

(array([[0., 2., 3.],
       [1., 0., 4.],
       [2., 3., 0.]]), array([[ -9999,    0,    0],
       [    1, -9999,    0],
       [    2,    2, -9999]], dtype=int32))
```

bellman ford- In this also we use and we find the shortest path using between all the pairs of elements

```
In [69]: import numpy as np
from scipy.sparse.csgraph import bellman_ford
from scipy.sparse import csr_matrix
new_value= np.array([[1,2,3],[1,0,0], [2,3,4]])
value_new= csr_matrix(new_value)
print(bellman_ford(value_new, return_predecessors=True, indices=0))

(array([0., 2., 3.]), array([ -9999,    0,    0], dtype=int32))
```

```
In [74]: import numpy as np
from scipy.sparse.csgraph import bellman_ford
from scipy.sparse import csr_matrix
new_value= np.array([[1,2,3],[-1,0,0], [-2,3,4]])
value_new= csr_matrix(new_value)
print(bellman_ford(value_new, return_predecessors=True, indices=0))

(array([0., 2., 3.]), array([ -9999,    0,    0], dtype=int32))
```

In []:

In []:

In []: